# Deep Learning for Perception - Assignment 2

Convolutional Neural Network for Image Colorization

Ibrahim Johar Farooqi - 23K-0074

## Part 1: Colorization as Regression

In this approach, image colorization is treated as a regression problem. The CNN takes a grayscale image as input and directly predicts the RGB pixel values at every location. Since the output is continuous, the model is trained using Mean Squared Error (MSE) as the loss function.

### Q1 - Convolution Layers

**Number of Layers**

The regression-based CNN (RegressionCNN) has 6 convolutional layers in total. The architecture follows an encoder-decoder structure where the first two layers progressively downsample the spatial resolution, a middle layer then processes features at the bottleneck, and the final three layers upsample back to the original 32×32 resolution.

**Architecture Diagram**

The data flows through the network as shown below:

**Grayscale Input (1×32×32)**

↓

***downconv1*** → Conv(1 → 32) + BN + ReLU + MaxPool → (32×16×16)

↓

***downconv2*** → Conv(32 → 64) + BN + ReLU + MaxPool → (64×8×8)

↓

***rfconv*** → Conv(64 → 64) + BN + ReLU → (64×8×8)

↓

***upconv1*** → Conv(64 → 32) + BN + ReLU + Upsample → (32×16×16)

↓

***upconv2*** → Conv(32 → 3) + BN + ReLU + Upsample → (3×32×32)

↓

***finalconv*** → MyConv2d(3 → 3) → RGB Output (3×32×32)

**Layer Summary Table**

| Layer | Input Channel | Output Channel | Kernel | Filters | Extra |
|---|---|---|---|---|---|
| **downconv1** | 1 (grayscale) | 32 | 3×3 | 32 | MaxPool2d |
| **downconv2** | 32 | 64 | 3×3 | 64 | MaxPool2d |
| **rfconv** | 64 | 64 | 3×3 | 64 | - |
| **upconv1** | 64 | 32 | 3×3 | 32 | Upsample ×2 |
| **upconv2** | 32 | 3 | 3×3 | 3 | Upsample ×2 |
| **finalconv** | 3 | 3 | 3×3 | 3 | MyConv2d |

MyConv2d is a custom implementation of Conv2d that manually defines weights and bias parameters. It performs the same 2D convolution operation as nn.Conv2d, but is written from scratch to demonstrate understanding of the convolution operation.

## Q2 - Training Epochs

**Default Training Configuration**

The CNN model was trained using the following default configuration:

- Optimizer: Adam
- Learning rate: 0.001
- Batch size: 100
- Default epochs: 25
- Loss function: Mean Squared Error (MSE)

**Training Variations - Results**

The model was trained five times with different numbers of epochs to study the effect of training duration on performance. Results are mentioned below:

| Epochs | Train Loss | Val Loss | Observation |
|---|---|---|---|
| **5** | 0.0150 | 0.0146 | Low and blurry, washed-out colors |
| **25** | 0.0088 | 0.0088 | Good, clear structure, decent color |
| **50** | 0.0068 | 0.0066 | Better, more defined, richer color |
| **75** | 0.0067 | 0.0065 | Marginal gain, near plateau |
| **100** | 0.0060 | 0.0067 | Overfitting, val loss rises |

## Analysis and Discussion

### Observation 1 - Diminishing Returns:

The most significant improvement occurs in the early stages of training. Over 5 to 25 epochs, the validation loss drops by approximately 0.006, which is a substantial gain. However, from 25 to 50 epochs, the improvement is only 0.002, and from 50 to 75 epochs, it shrinks further to just 0.001. This pattern of diminishing returns is typical in neural network training as the model captures the most important patterns early and makes only incremental improvements after that.

### Observation 2 - Overfitting at 100 Epochs:

At 100 epochs, the training loss (0.0060) becomes noticeably lower than the validation loss (0.0067). More tellingly, the validation loss stopped improving around epoch 83 and began to plateau and slightly rise. This is a clear sign of overfitting: the model starts to memorize the training data rather than learning generalizable features. The train-val gap, which was essentially zero at 5 and 25 epochs, has now opened up, which is a classic overfitting signature.

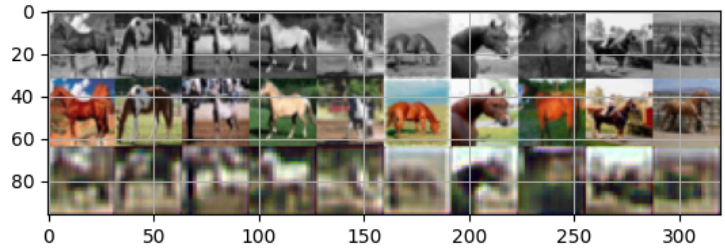
### Observation 3 - Optimal Training Duration:

Based on these results, the optimal training range for this model and dataset is approximately 25 to 50 epochs. This range achieves strong performance without signs of overfitting, and the training time remains reasonable. Training beyond 50 epochs yields diminishing improvements at the cost of longer training time and increasing risk of overfitting.

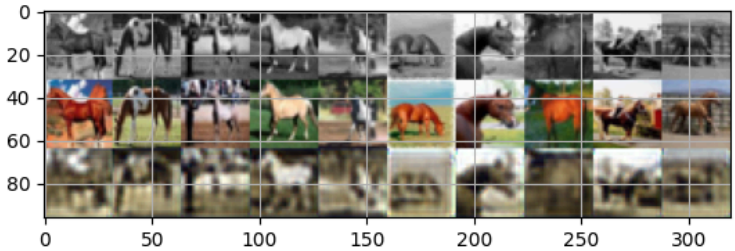
### Observation 4 - Visual Quality:

The 5-epoch output images appear washed out and predominantly grey; the model has not yet learned meaningful color associations. By 25 epochs, recognizable horse colors emerge, and at 50 epochs, the images show more defined color regions. Beyond 50 epochs, visual improvement becomes very hard to distinguish by eye despite continued numerical improvement in loss. This supports using loss as the primary metric for the stopping criterion rather than purely subjective visual inspection.

**5 Epoch**

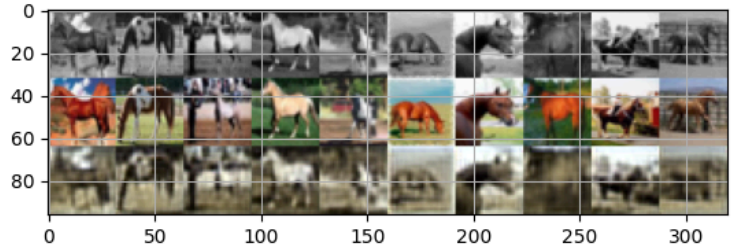



**25 Epoch**




**50 Epoch**

**75 Epoch**




**100 Epoch**

# Part 2: Colorization as Classification

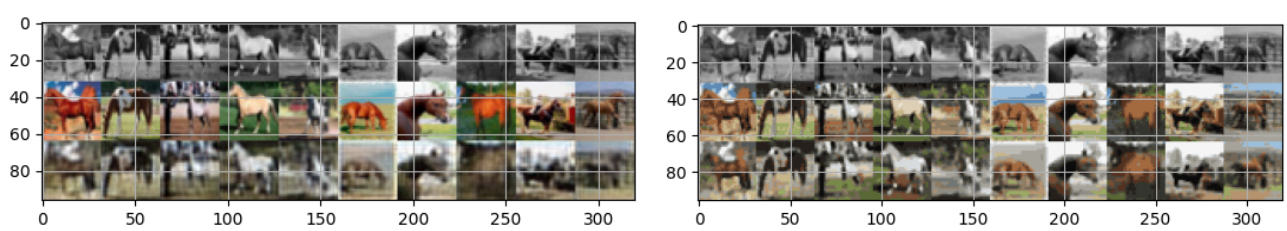
## Generating and Comparing Images

The regression model treats colorization as a pixel-wise prediction problem, outputting exact RGB values for each pixel. The classification model instead selects from 24 discrete color clusters obtained via k-means clustering, treating each pixel's color as a class label. Looking at the output images from both models, the following differences are observed:

- The regression model produces smoother color transitions across the horse's body, but the colors appear muted and desaturated, browns and tans blend into greyish tones because the model averages possible color values to minimize MSE loss. When multiple possible colors exist for a pixel, the model is constrained to predict an average, which is mathematically optimal for MSE but visually produces dull, washed-out results.
- The classification model produces more vibrant and saturated colors. Browns, blacks, and tans appear more distinctly because the model commits to one specific color cluster per pixel rather than averaging. However, the limited 24-color palette often causes color blocking artifacts, visible hard boundaries where adjacent pixels snap to different discrete colors.

## Did the classification model achieve better validation loss and accuracy?

The two models cannot be directly compared on loss values since they use fundamentally different loss functions, MSE for regression and cross-entropy for classification. However, the classification model introduces a meaningful accuracy metric (41.6% validation accuracy at 25 epochs), which regression cannot provide. In terms of visual quality, the classification model produces more perceptually appealing results despite the color-blocking limitation, because vibrant, committed colors sometimes align better with how humans perceive image quality than smooth but desaturated averages.




*Regression Output (Left) vs. Classification Output (Right)*

| Metric | Regression CNN | Classification CNN |
|---|---|---|
| **Loss function** | MSE | Cross-entropy |
| **Output Channels** | 3 (RGB Values) | 24 (color probabilities) |
| **Final train loss** | 0.0088 | 1.5629 |
| **Final val loss** | 0.0088 | 1.5836 |
| **Val accuracy** | N/A | 41.6% |

# Part 3: Skip Connections

## Did skip connections improve validation loss and accuracy?

Yes, the UNet with skip connections showed considerable gains against the plain classification CNN across every metric:

| Metric | Classification CNN | UNet |
|---|---|---|
| **Final train loss** | 1.5629 | 1.2963 |
| **Final val loss** | 1.5836 | 1.3062 |
| **Val accuracy** | 41.6% | 50.0% |

The UNet achieved 50% validation accuracy against CNN's 41.6%, which is an improvement of 8.4 percentage points. The validation loss dropped by approximately 0.28. Beyond the final numbers, the learning speed also improved quickly. UNet reached 40% accuracy by epoch 5 while the plain CNN only reached that at epoch 19, demonstrating that skip connections help the network learn meaningful color patterns much earlier in training.

## Did skip connections lead to qualitatively better outputs?

At 32×32 resolution, visual differences between the two models are modest but present. The UNet output shows more consistent color assignment across the horse body with less random color noise in background regions. Color boundaries appear slightly more defined, particularly around the horse silhouette. However, the quantitative metrics are a more reliable indicator of improvement than visual inspection at this low resolution; the 8.4% accuracy gap and 0.28 loss reduction confirm UNet is meaningfully better. At higher resolutions, the qualitative

improvement from skip connections would be far more pronounced, as sharper spatial detail would become visually distinguishable.

## Two reasons why skip connections enhance CNN performance:

**Reason 1 - Improved gradient flow during backpropagation:**
During training, gradients must travel backwards through every layer to update the network's weights. In deep networks without skip connections, this signal weakens as it passes through each successive layer, which is a problem known as the vanishing gradient problem. Skip connections create direct shortcut paths for gradients to flow from the loss function back to earlier layers without passing through every intermediate layer. This preserves the gradient signal and allows early encoder layers to receive meaningful updates, enabling more effective and faster training throughout the entire network.

**Reason 2 - Preservation of spatial detail through the bottleneck:**
MaxPooling in the encoder compresses spatial resolution to extract abstract, high-level features. By the time information reaches the bottleneck, fine spatial details like precise edges, object boundaries, and texture locations are largely lost, as the network has traded spatial precision for semantic understanding. Without skip connections, the decoder must reconstruct all of this spatial information from the compressed bottleneck representation alone, which leads to blurry color boundaries. Skip connections allow the decoder to directly access the encoder's feature maps at their original resolution. This means the network combines high-level semantic understanding from the bottleneck with low-level spatial precision from the encoder simultaneously, resulting in sharper and more accurate color boundaries in the final output.

## Visualizing intermediate activations

### 1. Visualizing CNN activations for test examples

**Conv1: Low-level feature detection**
In the first layer, the activations are still quite sparse overall, but they begin picking up simple local patterns such as edges, curves, and small texture changes. Rather than showing strong full-map activation, the feature maps contain scattered responses at places where the image has noticeable structure. This layer still retains enough spatial information for the object outline to be somewhat recognizable, but it is clearly focused on basic visual cues rather than higher-level meaning.

**Conv2: Broader pattern recognition**
By the second layer, the activations start becoming more selective and less detailed. Some feature maps remain active while others fade out, showing that the network is moving away from exact edges and toward larger visual regions. The object is still visible, but the representation is already becoming less precise and blurry.

**Conv3: Strong compression of information**
At the bottleneck, the image is reduced into a much smaller and more abstract representation. The activations no longer resemble the original object clearly, because most of the fine spatial detail has been removed. What remains is mainly high-level information that the network will use for reconstruction.

**Later layers: Rebuilding the image**
After the bottleneck, the decoder starts trying to recover the missing structure. The activations become broader and begin forming more meaningful regions again, but they are still somewhat blurry and not sharply defined. This shows that the network is reconstructing from limited compressed information.

**Conv5: Final color prediction maps**
In the last layer, the network outputs color probability maps at full resolution. Each channel represents a possible color choice, and the final color is selected from these probabilities. The output is useful, but the prediction is still somewhat soft around boundaries because the earlier compression removed a lot of detail.

**2. Visualizing Colorization UNet activations for test examples**

Compared with the CNN, the UNet keeps spatial structure much better in the deeper layers because the skip connections pass encoder features directly to the decoder. The early encoder activations are still similar to the CNN, since both networks respond to edges and textures, but the later layers in UNet look more organized and less heavily compressed. This helps the network preserve boundaries and recover finer detail, so the final color maps appear cleaner and more sharply separated than in the plain CNN. Overall, the UNet is better at combining global semantic information with local detail, which leads to more precise colorization results.

# Part 4: Theoretical Questions

## Q1 - Improving evaluation beyond pixel-level metrics:

Pixel-level loss functions such as MSE and cross-entropy evaluate images by comparing individual pixels in isolation. This approach has a fundamental limitation that an image can score well on pixel-level metrics yet still appear blurry, desaturated, or unnatural to a human observer. This disconnect exists because human perception evaluates images holistically, relative to structure, texture, edges, and coherence rather than individual pixel values.

A more effective alternative is perceptual loss, introduced by Johnson et al. (2016) [2]. Instead of comparing raw pixel values, both the predicted and ground truth images are passed through a pretrained network such as VGG-16, and their intermediate feature representations are compared. These feature maps capture perceptually meaningful information like structure, texture, and semantic content that aligns much more closely with human visual judgment. Minimizing perceptual loss typically produces sharper, more visually coherent results than minimizing MSE.

Additional metrics that better reflect human perception include SSIM (Structural Similarity Index), which measures luminance, contrast, and structural similarity simultaneously, and LPIPS (Learned Perceptual Image Patch Similarity), which uses deep features to measure perceptual distance between images. Human evaluation studies, where users rate or compare colorized images directly, represent the gold standard for perceptual quality assessment but are expensive and difficult to scale.

## Q2 - Adapting trained models to colorize images larger than 32×32:

The trained models are fully convolutional as they contain no fully connected layers. Every operation (convolution, batch normalization, pooling, and upsampling) is spatially flexible and can operate on inputs of any size. This means the network can, in principle, accept larger images without architectural changes.

However, two practical considerations must be addressed. First, the total downsampling factor of the network is 4 (two MaxPool2d layers, each with stride=2). The input image dimensions must therefore be multiples of 4. If the input image is not a multiple of 4, it should be padded to the nearest multiple before processing and then cropped back afterward.

Second, for very large images, memory constraints may require a sliding window approach; dividing the image into overlapping 32×32 patches, colorizing each patch independently, and then stitching the outputs together by averaging overlapping regions to avoid visible seams at patch boundaries.

In practice, the simplest approach is to pad the large image to the nearest multiple of 4, pass the entire image through the network in one forward pass, and crop back to the original dimensions. The network will produce an output of the same spatial size as the input since the upsampling exactly reverses the downsampling.